CampusFlow Web Service Report

Name: ZHU FUXIN

Student ID: M25W7195

Assignment: Web service client, account authorization, Docker, and Azure deployment

1. Project overview

CampusFlow is a desktop browser client backed by a Spring Boot web service. The project combines a designed frontend with external information services, database persistence, account authorization, user-owned portfolio storage, and container deployment.

The final client is organized as three pages:

1. Today is the default page. It displays Kyoto preset content immediately, then updates weather and country information from the backend.
2. Portfolio is a public personal page for photography, drawing, design, writing, audio, and project work. Guest data is available before login.
3. Account supports local registration and login as the primary method. Google and GitHub provide optional third-party login.

English, Japanese, and Simplified Chinese can be switched from the same language control. The visual system is shared by all three pages, while the composition of each page is intentionally different.

2. System architecture

The application uses:

- HTML, CSS, JavaScript, and GSAP for the browser client
- Java 17 and Spring Boot for the web service
- Spring Security for OAuth, CSRF, sessions, and password encoding
- MySQL 8 for the local Docker database
- H2 for automated tests, the classroom launcher, and Azure App Service storage
- Docker and Docker Compose for reproducible local execution
- Docker Hub and Azure App Service for the online container

The browser and API are served from one Spring Boot application. This avoids a

separate CORS configuration and keeps both local and cloud execution easy to demonstrate.

3. Main web services

Daily information

`GET /api/home` receives a country and city, calls REST Countries and Open-Meteo, and returns one combined response. The weather request uses the timezone returned by geocoding, so the displayed date and time belong to the selected city rather than the server timezone.

Profile

`GET /api/profile` returns a public profile. Student ID and phone are excluded for visitors. An authenticated owner can update their profile and usual location through `POST /api/profile`. The identity title is selected from a fixed list used consistently by the English, Japanese, and Chinese interfaces.

Portfolio and media

`GET /api/portfolio` returns either the preset guest works or the current user's portfolio. Authenticated users can create, update, and delete only their own items.

A work can be created without a file or uploaded as an image, audio, or text file. The file is stored outside the database with a generated filename. The database stores its owner, original filename, content type, stored filename, size, visibility, display order, and presentation settings.

The editor provides three card sizes: `STANDARD`, `WIDE`, and `TALL`. Images can show their complete frame with `CONTAIN` or fill the selected card with `COVER`. These settings remain attached to the work after later edits.

Accounts

`POST /api/register` creates a normal `USER` account.
`POST /api/authenticate` verifies a local password and issues an HttpOnly session cookie. `GET /api/auth/me` reports the current public account state, and `POST /api/auth/logout` removes the session.

Google and GitHub OAuth use the same account model. After authorization, the provider name, display name, and avatar are shown in the Account and Portfolio

pages. The provider request opens the account picker so another visitor can choose their own account.

An `ADMIN` account is never created by public registration. It is provisioned only through server environment variables and uses `/api/admin/users` to review, enable, disable, or remove ordinary accounts.

4. Security design

- Local passwords are encoded with BCrypt strength 12.
- Passwords are never returned by an API.
- Password fields are masked in the browser.
- The session token is stored in an HttpOnly cookie, not browser local storage.
- Azure enables the Secure cookie flag.
- State-changing API calls include a CSRF token.
- Server-side ownership checks protect profile, media, and portfolio writes.
- Public registration cannot request the administrator role.
- OAuth secrets and administrator credentials are environment variables.
- Uploads are limited by file category, extension, size, and signature.
- Generated storage names and normalized paths prevent path traversal.

5. API summary

Method	Path	Purpose
GET	`/api/home`	Combined weather and country data
GET	`/api/profile`	Public or current-owner profile
POST	`/api/profile`	Save the signed-in owner's profile
GET	`/api/portfolio`	Public or current-owner works
POST	`/api/portfolio`	Create an owned work without a file
POST	`/api/portfolio/upload`	Upload image, audio, or text work
GET	`/api/portfolio/{id}/media`	Read an authorized stored file
PUT	`/api/portfolio/{id}`	Update an owned work
DELETE	`/api/portfolio/{id}`	Delete an owned work
POST	`/api/register`	Create a normal local account
POST	`/api/authenticate`	Sign in with username and password
GET	`/api/auth/me`	Read current account state
POST	`/api/auth/logout`	Sign out

Method	Path	Purpose
GET	`/api/oauth/status`	Read third-party authorization state
GET	`/api/admin/users`	Administrator account management
PATCH/DELETE	`/api/admin/users/{id}`	Change or remove an ordinary account

Interactive API documentation is available at:

```
http://localhost:8080/api-docs.html
```

6. Local execution

The Dockerfile uses a multi-stage build. Maven compiles the project and runs the automated test suite before the runtime image is produced. Docker Compose starts:

- `campus-flow-app` on port `8080`
- `campus-flow-mysql` as the MySQL sidecar

The application connects to MySQL with the Compose service name `mysql`. Secrets can be supplied from a local `.env` file, which is excluded from Git.

Run the Docker environment with:

```
docker compose up -d --build
```

The compiled submission can also run without Docker. `START_LOCAL.cmd` starts `output/submission/CampusFlow.jar` with the `classroom` profile and stores its H2 database and uploaded media under the project `data` directory.

7. Azure deployment

The final image is:

```
berhish/campus-flow:release-20260727-oauth
```

The only Azure target is:

```
Web App: campusflow-final-0724
URL: https://campusflow-final-0724-grbzdrczdqdyhyea.japanwest-01.azurewebsites.net
Container port: 8080
```

Azure uses the production H2 profile because the Free F1 plan does not provide a MySQL sidecar. The H2 file and uploaded media are stored below `/home/campusflow`, where App Service storage preserves them across normal container restarts. Google and GitHub callback URLs are derived from

`APP_BASE_URL`.

8. Verification

The 16 automated regression tests check:

- public registration cannot create an administrator
- local passwords are BCrypt hashes
- the configured administrator follows the current environment values
- an old signing secret cannot authorize an administrator request
- unauthenticated profile writes are rejected
- authenticated profile saves succeed
- profile identity titles are restricted to the preset list
- a new account begins with an empty portfolio
- an item created without a file does not receive a placeholder image
- uploaded media remains readable after its metadata is edited
- card size and image fit settings remain persisted
- guest portfolio contains distinct preset images
- private profile fields are hidden from visitors
- English, Japanese, and Chinese are included in the release UI
- Google and GitHub redirects use the configured public base URL
- weather uses the selected city's timezone

The final browser acceptance also checks page layout, guest presets, masked password input, language switching, ownership, third-party account selection, media persistence, and account state. The Azure service was accepted at its published HTTPS address.

9. Conclusion

CampusFlow demonstrates a complete browser client and Spring Boot web service with external data, secure accounts, owner-scoped portfolio storage, Docker, and one Azure container. The project can be run from the compiled JAR, rebuilt with Docker Compose, or reviewed through the published service.